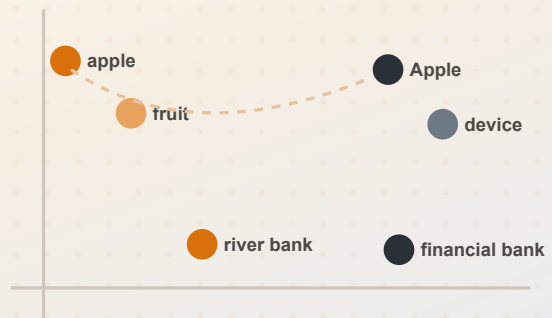


How machines represent meaning

Meaning is positioned through learned relationships



AI NAMASTE AI - SEASON 1

Episode 05

How machines represent meaning

This episode follows the path from meaningless token IDs to learned embeddings, then shows how dimensions, similarity, position, context, and practical retrieval systems help machines work with relationships in language and other data.

02:25:36

WebVTT transcript only

In this episode

01 Why token IDs do not contain meaning

02 From vectorization to learned embeddings

03 Dimensions, coordinates, and vector neighborhoods

04 Semantic and cosine similarity

05 Token, text, positional, and contextual representations

06 Bias and the limits of similarity

07 Search, recommendations, clustering, and RAG

08 Multimodal embeddings and final misconceptions

When the same word means different things

00:00 - 07:16

Consider two uses of the same word:

- “I ate an apple in the afternoon.”
- “Apple has launched a new device.”

In the first sentence, *apple* refers to a fruit. In the second, *Apple* refers to a company. The same problem appears with *bank*: a person can sit near a riverbank, or keep money in a financial bank. Tone and sarcasm add another layer because even the same words can communicate different intentions.

The episode begins with a deceptively simple question: how can a machine tell which meaning is intended?

Q: Does a token ID tell the model what a token means?

A: No. A token ID identifies an entry in a tokenizer's vocabulary, but the number itself does not describe the token's meaning.

The lecture revisits the earlier pipeline: a sentence is split into tokens, and each token is mapped to an ID. To test what those IDs really tell us, the instructor invents four dummy values:

Token	Dummy token ID
dog	8123
mango	612
cat	123
grapes	8521

These values are explicitly made up for the demonstration; they are not actual tokenizer IDs. If the words disappear and only the numbers remain, nothing in 8123 tells us that *dog* is related to *cat*. In fact, 8123 and 8521 look numerically close, yet *dog* and *grapes* do not have the conceptual relationship the exercise is looking for.

Two everyday analogies reinforce the point. A student's roll number does not reveal whether that student is studious or similar to the student with the next number. In the same way, neighboring hotel rooms such as 102 and 103 are close as labels, not as meanings.

Vectorization: turning information into numbers

07:16 – 12:13

The bridge from an identifier to something mathematically useful is **vectorization**. In the lecture's plain-language definition, vectorization means converting information into a numerical vector, and a vector can be thought of as an array of numbers.

The input does not have to be a single word. The instructor names many kinds of information that can be vectorized:

- a word, sentence, paragraph, or document;
- an image or video;
- a JSON object;
- code;

- a product or another item;
- any other information a computer needs to process numerically.

Q: Why convert text, images, or other information into numbers?

A: Because a computer can perform mathematical operations on numerical representations. A random array does not automatically contain meaning, but numbers give the system something it can calculate with.

This distinction is worth holding onto: **vectorization provides a numerical form; learning is what shapes that form into a useful representation.**

A three-dimensional fruit exercise

12:13 – 21:45

To make the idea of dimensions visible, the instructor asks learners to score edible items using three properties: sweetness, size, and crunchiness. The values are illustrative and subjective, not measurements from an embedding model.

Item	Sweetness	Size	Crunchiness
apple	0.7	0.4	0.8
banana	0.8	0.6	0.2
carrot	0.2	0.5	0.9
watermelon	0.6	0.9	0.5 or 0.6

Watermelon's crunchiness is spoken as both 0.6 and 0.5 or six. The transcript does not establish one final value.

The instructor also notices that a carrot does not belong cleanly in a fruit-only list, then reframes the exercise around edible items or suggests restricting it to fruits. The classification is not the lesson. The lesson is that one item can be described by several numeric coordinates.

Each value is one **dimension** in this teaching example:

- dimension 1: sweetness;
- dimension 2: size;
- dimension 3: crunchiness.

Real embeddings may contain hundreds or thousands of learned dimensions. Crucially, the instructor later warns that real dimensions are not individually labelled *sweetness*, *size*, or *crunchiness*. Those names make the idea easy to picture, but learned meaning is distributed across many dimensions.

A small vectorization exercise

Three properties turn each item into a three-dimensional numeric vector

Item	Sweetness	Size	Crunchiness
apple	0.7	0.4	0.8
banana	0.8	0.6	0.2
carrot	0.2	0.5	0.9

Illustrative, human-assigned dimensions - not learned embedding values

The fruit exercise turns three human-assigned properties into a small numerical vector. It is an analogy; real embedding dimensions are learned rather than given these fixed labels.

What makes a vector an embedding?

21:45 – 30:01

The lecture's central definition is:

EMBEDDING

A learned numerical representation of an item that captures useful relationships with other items.

The word **learned** separates an embedding from an arbitrary array of numbers. An **embedding model**, in the instructor's wording, takes information and produces embeddings. During training, repeated patterns in large amounts of data gradually influence the numerical values.

The instructor builds the intuition with sentences such as:

- "A banana is a sweet fruit."
- "I ate a sweet banana."
- "My kids like bananas because they are sweet."

As related words repeatedly appear in related linguistic environments, the model's values are adjusted. Over many examples, useful relationships emerge. This is a high-level story of learning, not a literal per-dimension update recipe.

Three four-dimensional examples then make the relationship visible:

```
king = [0.81, 0.32, 0.52, 0.17]
queen = [0.79, 0.36, 0.48, 0.22]
banana = [exact on-screen vector not available in the VTT]
```

The `king` and `queen` values are close dimension by dimension: `0.81/0.79`, `0.32/0.36`, `0.52/0.48`, and `0.17/0.22`. Banana's displayed values are described as farther away. The point is not that these four numbers are a literal definition of royalty; it is that their numerical relationship can capture something useful.

Learned neighborhoods and vector space

30:01 - 40:11

As training shapes embeddings, related concepts can form **neighborhoods** or **clusters**. The lecture names several possible groups:

- `king`, `queen`, `crown`, `castle`, `royalty`, and `kingdom`;
- `apple`, `banana`, and other fruit-related terms;
- `sun`, `moon`, `earth`, and `planet`;
- `JavaScript`, `Python`, and programming-language terms.

The instructor compares this with a child repeatedly hearing stories about kings, queens, crowns, and castles. Repeated exposure builds associations. A second, more playful analogy imagines adults calling every monkey a “king” in front of a newborn; the child would learn from the usage it encounters. In both analogies, the supplied data matters.

The multi-dimensional arrangement of learned vectors is called a **vector space**. An embedding can be treated like a coordinate in that space. Because people can comfortably view only two or three dimensions, the instructor reduces the idea to a 2D plot, placing king near queen, man near woman, apple near banana, and JavaScript near Python.

The lecture also points to a directional pattern: the direction from man to woman is described as resembling the direction from king to queen. The exact plotted coordinates are not recoverable from the transcript.

IMPORTANT NOTE

An embedding is not a human mental image or a dictionary definition. It is a learned numerical representation of patterns and relationships.

How many dimensions are enough?

40:11 - 54:39

The fruit exercise used three dimensions, but language requires far richer representations. To explain why, the instructor first describes a person using only height and weight. Two coordinates omit age, profession, location, skills, interests, experience, language, ethnicity, religion, and many other aspects. Adding dimensions can capture more of that complexity.

Language adds its own difficulties:

- one word can have several senses;
- many languages must be represented;
- grammar may be imperfect;
- tone and sarcasm can reverse or reshape the intended meaning;
- a one-off co-occurrence does not necessarily create a durable relationship.

Examples include **Apple** as fruit or company, **Java** as a programming language, coffee, or an island, and the phrase “terribly good,” where the instructor explains *terribly* as participating in an expression meaning “extremely good.”

More dimensions can represent more complex patterns, but the episode rejects a simple equation between size and intelligence. Larger representations also demand more storage and computation.

More dimensions do not automatically make a model smarter.

The instructor says vector neighborhoods emerge from patterns in data rather than being manually programmed. Example statements include “The king ruled the kingdom,” “The queen addressed the nation,” “JavaScript is used in web development,” “Python is popular in machine learning,” and “Apples and bananas are fruit.” He connects this learning process to the importance of data and compute.

Some comments in this portion concern public internet data, artists’ objections, stylistic image generation, and competition around data and compute. They are presented as the instructor’s commentary, not independently verified claims.

[Transcript unclear] A short transition around **31:10–32:17** is partially garbled. The surrounding cues clearly preserve the ideas that values are learned rather than assigned dimension by dimension and that training requires compute.

Around **50:33–50:43**, the instructor refers to numeric adjustment as “tuning” and then says “fine tuning the data.” The phrase is retained here only as lecture wording and is not formalized into a separate process.

Semantic similarity: looking beyond exact words

54:39 – 01:04:38

Semantic similarity is introduced as closeness in meaning or intent between two pieces of text. The key comparison is:

Query A

“How do I center a div?”

Query B

“How can I align an HTML element in the middle of its parent?”

The word overlap is small, but the instructor says the intent is the same. A system that relies only on exact keywords may miss that relationship. If each complete query receives a text embedding, their learned vectors can be compared instead.

Other pairs reinforce the same idea:

- “How can I reset my password?” and “I forgot my password. How do I create a new one?”
- “The application crashes after login.” and “The software closes immediately when I sign in.”

The opposite failure mode is also important:

- “How do I learn Java?”
- “How do I make Java coffee?”

Both contain *Java*, yet their meanings differ. Exact overlap alone is therefore not enough.

[Transcript unclear] Captions around 54:45–55:17 are severely corrupted, but the surrounding explanation clearly establishes semantic similarity and the CSS-centering example.

Cosine similarity: comparing vector direction

01:04:38 – 01:18:16

The episode uses **cosine similarity** to compare vectors. The instructor focuses on their direction—the angle between them—more than their absolute lengths. Two arrows can be different lengths yet point in roughly the same direction.

The spoken scale is:

- angle near 0° → cosine near 1 → similar direction;
- angle near 90° → cosine near 0 → orthogonal direction;
- angle near 180° → cosine near -1 → opposite direction.

The formula is described as the dot product divided by the product of the vectors’ magnitudes:

$$\text{similarity}(A, B) = (A \cdot B) / (\text{magnitude}(A) \times \text{magnitude}(B))$$

The lecture does not calculate a numerical example. It explains the purpose of the numerator, magnitudes, and resulting scale at a high level.

At 01:07:51, a value of zero is described as “somewhat similar,” while the surrounding explanation calls the 90° case orthogonal or unrelated. The transcript contains both phrasings.

An article about cosine similarity on [LearnDataSci.com](#) is mentioned. [Incomplete URL] The transcript supplies only the domain and topic, not the exact page address.

Cosine similarity

The lecture compares vector direction rather than absolute length



Cosine similarity compares vector direction: the lecture connects angles near 0° , 90° , and 180° with cosine values near 1 , 0 , and -1 .

Similarity is not truth

Two statements can be close in subject while disagreeing completely. The instructor uses:

- “JavaScript is the best language.”
- “JavaScript is the worst language.”

Both discuss an opinion about JavaScript, so their embeddings may be related even though their judgments are opposite. The same limit applies to false statements and harmful instructions.

VERY IMPORTANT

Embedding similarity does not prove truth, quality, safety, intent, or agreement. Embeddings capture relationships; they do not judge them.

Seeing embeddings in a projector

01:18:16 – 01:29:17

The instructor opens [projector.tensorflow.org](#), described as TensorFlow’s Embedding Projector, and searches for terms including [sun](#), [king](#), [JavaScript](#), [Apple](#), and [bank](#). Nearby words and visible groupings provide an intuitive view of the learned space.

For [sun](#), the surrounding terms include [moon](#), [solar](#), [sky](#), and [eclipse](#). For [king](#), the lecture observes nearby royalty-related words. [JavaScript](#), [Apple](#), and [bank](#) demonstrate that a word’s neighborhood can reflect several kinds of association.

The instructor explicitly says he is not sure how the displayed distances are calculated. That uncertainty should not be replaced with a mechanism the lecture does not provide.

PROJECTION, NOT THE ORIGINAL SPACE

A 2D or 3D projector is a human-viewable reduction of a much higher-dimensional embedding space. It helps with intuition, but it is not a perfect display of the original geometry.

Embedding of what-and for which task?

01:29:17 – 01:44:45

Embeddings are not limited to individual tokens. The instructor names tokens, sentences, paragraphs, documents, PDFs, images, audio, and video as things that can receive vector representations.

The phrase “I love JavaScript” is first processed token by token. Every token has an identity and receives a token embedding. Yet token identity alone is not enough because order changes meaning:

Order 1

dog bites man

Order 2

man bites dog

The same three token identities appear in both sentences, but their roles and overall meanings differ. The lecture therefore separates two signals:

- **token embedding** helps identify which token is present;
- **positional information** helps identify where it occurs.

No exact positional-encoding formula is given. The architecture may supply position in different ways; the learner-facing point here is simply **identity + order**.

A **text embedding** is one vector for a larger piece of text, such as the complete sentence “I love JavaScript.” In the lecture, text embeddings support search, retrieval, clustering, recommendation, classification, and duplicate detection.

Representation	Lecture purpose
Token embedding	Helps an LLM process individual tokens and their sequence internally.
Text embedding	Helps an application compare or retrieve larger pieces of content.

The instructor’s decision question is more useful than memorizing one definition: **embedding of what, and for which task?**

[Transcript unclear] Short intervals around 01:41:20–01:41:50 and 01:42:49–01:43:19 are corrupted. The surrounding cues support the token-versus-text distinction above, but the intervening wording cannot be reconstructed confidently.

Context changes the representation

01:44:45 – 01:54:34

A static embedding assigns one fixed initial vector to a token. That creates a problem for **polysemy**, the lecture's term for one word having multiple meanings.

Examples include:

- **Apple** : fruit / company;
- **bank** : river edge / financial institution;
- **Java** : programming language / island / coffee;
- **bat** : object used to hit a ball / animal;
- **state** : political region / solid, liquid, or gaseous state / a person's condition.

Context means the surrounding words or data that help reveal the intended sense. Words such as *ate* and *lunch* support the fruit sense of Apple, while *released* and *device* support the company sense.

At a high level, the transformer-based process in the lecture is:

- 1 The token starts with the same initial token embedding.
- 2 The model processes it together with the surrounding tokens.
- 3 Across multiple transformer layers, the internal representation is modified.
- 4 The result becomes context-specific: a financial-bank representation or a river-bank representation.

The instructor calls this **contextualization** and reduces it to one memorable sentence: **context modifies the representation**.

[Transcript unclear] Captions around 01:53:54–01:54:17 are corrupted. The cues immediately before and after them preserve the conclusion above.

When data patterns include bias

01:54:34 – 02:00:40

Training data is described as largely human-created. That means it can contain cultural patterns, historical inequalities, stereotypes, representation gaps, social biases, and conflicting interpretations of history or politics. An embedding may therefore encode undesirable associations along with useful ones.

The lecture's concrete example is a repeated gender-occupation association. If an occupation is repeatedly connected with one gender in the data, the learned representation may also carry that pattern. The instructor also names race or colour, religion, historical figures, and other sensitive topics as areas that require care.

The point is not that an embedding consciously chooses prejudice. It is that a representation learns patterns from its data, including patterns a designer may not want reproduced. The instructor says AI companies have a social responsibility to make systems safe and avoid promoting injustice, inequality, and stereotypes.

Semantic search and hybrid search

02:00:40 – 02:03:42

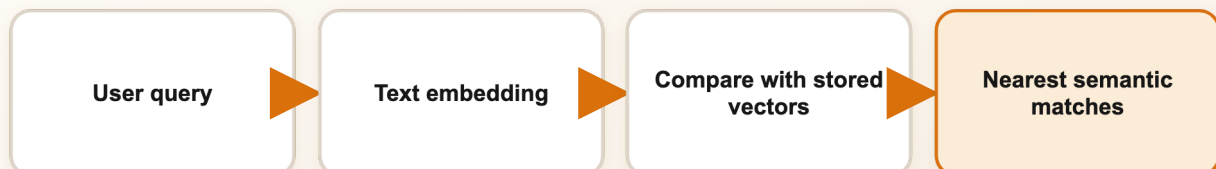
For semantic search, the episode gives a compact workflow:

- 1 Convert a query and the candidate text into embeddings.
- 2 Compare their vectors, including with cosine similarity.
- 3 Retrieve material that is close in meaning.

Exact keyword search still matters. A product ID, error code, date, place, or specific legal clause may need a literal match. The instructor therefore introduces **hybrid search** as a combination of keyword search and embedding-based semantic search.

Semantic search flow

Compare learned meaning representations, not only identical keywords



Semantic search compares learned representations of the query and candidate text. The lecture keeps exact keyword matching for identifiers, codes, dates, places, and legal clauses, and calls their combination hybrid search.

The lecture broadly says that “modern LLMs use both,” without identifying a particular system. Treat this as the instructor’s general statement, not as a verified universal implementation detail.

Recommendations, clustering, RAG, and classification

02:03:42 – 02:14:51

Recommendation systems

The instructor imagines a vector for a video's content and a second vector for a user's viewing interests. Comparing them can help identify potentially relevant videos. A video about the event loop may relate to a search for the microtask queue even when the wording differs.

YouTube, Spotify, LinkedIn, and Twitter/X are named as recommendation examples. The instructor immediately adds a caution: embeddings are not the entire YouTube algorithm; he is presenting them as one possible component.

Clustering

Vectors can help group similar items, including student questions, YouTube comments, emails, or emails related to finance. The rough intuition is that nearby vectors can be collected into related groups. The lecture acknowledges that a full implementation is more involved.

Retrieval-augmented generation

RAG-**retrieval-augmented generation**-is introduced here, while its complete treatment is deferred to a later season. The example uses a 2,000-page physics book:

- 1 Divide the book into smaller chunks, perhaps a page or half-page at a time.
- 2 Create one embedding for each chunk.
- 3 Create an embedding for "What is thermodynamics?"
- 4 Find the chunk vector closest to the query vector.
- 5 Use that retrieved chunk as the likely answer context.

The episode defines **chunking** as dividing data into smaller pieces.

Duplicate detection and classification

Close vectors can help detect duplicate content. They can also support classification. The support-ticket buckets named in the lecture are:

- billing or invoice;
- technical issues such as a video not opening, mobile-app trouble, or login trouble;
- refund request;
- authentication or access issues.

Multimodal embeddings

02:09:33 – 02:14:51

Text is not the only modality in the episode. Images, photos, video, audio, documents, and paragraphs can also have embeddings. The main cross-modal example compares the text “**white cat**” with an image of a white cat. Their representations can be aligned so they occupy a similar region or direction, enabling text-to-image search and visually related recommendations.

Misconceptions to leave behind

02:14:51 – 02:25:36

The closing stretch revisits the most common misunderstandings.

Misconception

An embedding is a dictionary of tokens.

Each dimension has one fixed human-readable meaning.

Similar embeddings prove identical meaning or truth.

Lecture correction

An embedding is a learned numerical representation.

Meaning is distributed across learned dimensions.

Similarity establishes relatedness, not identity or truth.

The instructor adds three more corrections:

- No single embedding model is best for every task; the lecture says suitability depends on domain, data type, text length, and training objective.
- A 2D or 3D visualization is only a projection of the high-dimensional space.
- More dimensions involve trade-offs in storage, latency, cost, and quality; a larger vector does not guarantee a smarter model.

The final conceptual pipeline is:

text → tokens → token IDs → token embeddings → positional and contextual processing
→ vectors and similarity → retrieval and comparison tasks

The next episode is introduced as an exploration of neural networks and how models learn.

Chapter summary

Token IDs provide stable labels, but labels do not encode meaning. Vectorization converts information into a numerical form, and training shapes those numbers into embeddings that capture useful relationships. Related concepts can form neighborhoods in a high-dimensional vector space, while cosine similarity provides one way to compare vector direction.

The episode then moves beyond isolated vectors. Position distinguishes “dog bites man” from “man bites dog,” and context changes an initially shared token representation into a context-specific one. Similarity is powerful, but it is not a judgment of truth, agreement, quality, or safety, and learned representations can preserve unwanted patterns from their data.

Finally, embeddings are connected to semantic and hybrid search, recommendations, clustering, RAG, duplicate detection, classification, and multimodal retrieval. The recurring decision is always: **what is being embedded, and what task should that representation support?**

Key takeaways

- A token ID is an identifier; an embedding is a learned numerical representation.
- Embedding relationships emerge from patterns encountered during training.
- More dimensions can express richer patterns, but they also bring storage and compute costs.
- Cosine similarity compares vector direction, yet relatedness is not truth or agreement.
- Token identity, position, and context contribute different information.
- Embeddings can encode unwanted social and cultural associations from data.
- Practical uses include search, recommendations, clustering, RAG, classification, and multimodal retrieval.

Revision questions

1. Why can the dummy token IDs 8123 and 8521 not tell us that dog and grapes are related?
2. How do the roll-number and hotel-room analogies explain the role of token IDs?
3. What is vectorization, and why does the lecture say computers need it?
4. In the fruit exercise, what do sweetness, size, and crunchiness represent?
5. Why does the instructor later warn that real embedding dimensions do not each have one human-readable label?
6. State the lecture’s definition of an embedding. Why does the word *learned* matter?
7. Why are the sample king and queen embeddings treated as closer than king and banana?
8. What is a vector space, and why is a 2D or 3D projector only an approximation?
9. Why does more dimensionality not automatically mean more intelligence?
10. How does semantic similarity differ from exact keyword matching?
11. Why are “How do I learn Java?” and “How do I make Java coffee?” a failure case for naive keyword overlap?
12. What does cosine similarity compare, and how does the lecture describe values near 1, 0, and -1?
13. Why can “JavaScript is the best language” and “JavaScript is the worst language” still be semantically related?
14. What is the difference between a token embedding and a text embedding?
15. Why do “dog bites man” and “man bites dog” require positional information?
16. Define context, polysemy, and contextualization using Apple or bank.

17. How can patterns in human-created data become embedding bias?
18. When is keyword search preferable to embedding search, and what does hybrid search combine?
19. Walk through the episode's 2,000-page-book RAG example in order.
20. How can a text embedding for "white cat" be related to an image embedding for a white cat?